

A Network Optimization Framework for Portable Charging-Station Placement on the University of Wisconsin–Madison Campus

Linyan Li (lli643@wisc.edu)

Shirley Xu (mxu369@wisc.edu)

CS/ECE/ISyE 524 Final Project Report • Spring 2026

1 Introduction



Figure 1. Portable charging kiosks for rentable power banks.

This project formulates the placement of *portable charging kiosks* on the University of Wisconsin–Madison campus as a family of network-optimization problems and solves them in Julia/JuMP with the open-source HiGHS solver. Five progressively richer mixed-integer programs are developed, beginning with a basic minimum-cost shortest-path formulation and culminating in an equity-aware, budget-constrained model. Computational experiments on a graph of $|N| = 20$ buildings, $|E| = 32$ pedestrian edges, and $|R| = 36$ commuting origin–destination (OD) pairs identify a set of robust station locations and produce a three-phase deployment recommendation that achieves 100% coverage of all student trips with only five kiosks.

Code Availability. The full implementation, data, and visualization scripts are available at: <https://github.com/Linyan0115/uw-madison-charging-optimization.git>.

1.1 The Problem and Why It Matters

University students spend 8–12 hours a day moving between classes, dining halls, libraries, and residences. Their phones and laptops are used continuously throughout this time—for course materials, navigation, communication, and even payments—and surveys of U.S. undergraduates indicate that more than 90% of them consider a charged phone essential to daily life. “*Battery anxiety*”—the period between a low-battery warning and finding a place to charge—can extend well beyond an hour, disrupting class attendance, study sessions, and personal safety.

Portable charging stations (e.g., ChargeFUZE, Brick, FuelRod) are secure kiosks that dispense rental power banks via a mobile application. They offer a low-friction, low-footprint solution. Their deployment, however, is genuinely a planning problem: too few kiosks and coverage gaps persist; too many and the installation budget becomes infeasible. This decision is a natural application of optimization: identifying the smallest, cheapest, and most equitable set of locations that keeps every student’s device above a comfort threshold during their daily walks.

1.2 Brief History of the Underlying Problem Class

The placement problem is related to a class of *facility-location* models. One early example is Hakimi’s p -median problem [5], which sought switching-centre locations on a network so as to minimize total weighted distance to demand points. Church and ReVelle [4] introduced the *maximum-coverage location problem* (MCLP), which underlies our budget-constrained formulation.

The most direct methodological ancestor of our model is the *Flow-Refueling Location Model* (FRLM) of Kuby and Lim [6], refined by Capar et al. [2]. The FRLM observes that, in vehicle-refuelling settings, demand is generated along *paths* rather than at nodes, and that a vehicle with finite range must encounter a facility along its path frequently enough to avoid running out of fuel. Chan [3]—a Summer 2020 student in this same course—adapted the FRLM to site DC fast-chargers across Wisconsin’s interstate network, a project we explicitly draw inspiration from.

Our contribution is to translate that highway-scale framework to a *pedestrian campus network*: the “vehicle” is a student-carried device, the “range” is a battery charge, and the “path” is a shortest walking route between two campus buildings. The mathematical structure is isomorphic, but the parameter regime—meters versus miles, percent battery versus kWh—and the equity considerations make the campus problem genuinely different.

1.3 Where the Data Comes From

All node coordinates are real latitudes and longitudes manually collected from Google Maps for 20 representative UW–Madison buildings; these are the actual GPS positions a phone would record on campus. The pedestrian edges E were drawn based on the campus walkway network. Edge *lengths* are then computed analytically from the coordinates using a local equirectangular projection (Equation (2.1)). The 36 OD pairs were chosen by hand to reflect three realistic flow categories—residence-to-academic morning commutes, between-class transitions, and trips to community amenities. Installation-cost weights are calibrated to reflect known practical considerations (theft risk, available outlets, custodial coverage). The Monte Carlo robustness study additionally generates synthetic OD weights from Uniform(0.5, 1.5) to test whether the optimal solutions persist under demand uncertainty.

1.4 Simplifying Assumptions

Throughout this work we assume:

- (A1) students follow exact shortest paths between origin and destination;
- (A2) battery consumption is linear in walking distance ($\rho = 1$).
- (A3) kiosks have unlimited capacity (no queueing);
- (A4) a kiosk fully recharges any device that visits it (instantaneous jump to 100%);
- (A5) the network is undirected and demand is symmetric.

These are reviewed and partially relaxed in Section 4.8.

1.5 Outline

The overall workflow of this project is summarized in Figure 2. We begin by constructing a campus pedestrian network from real geographic data, and then develop a sequence of five progressively richer optimization models.

Section 2 presents the five mathematical models in standard form. Section 3 implements them in Julia/JuMP, with code blocks interleaved with the explanatory prose required by the rubric. Section 4 reports computational results, including sensitivity studies on three parameters, a Monte Carlo robustness study, a battery-trajectory visualization, and a side-by-side comparison against a simpler “Connectivity Mode” alternative. Section 4.8 discusses the limitations of our approach and the sensitivity of conclusions to our assumptions. Section 5 summarizes findings and proposes a concrete follow-up project—a two-stage stochastic program driven by Wi-Fi-association traces.

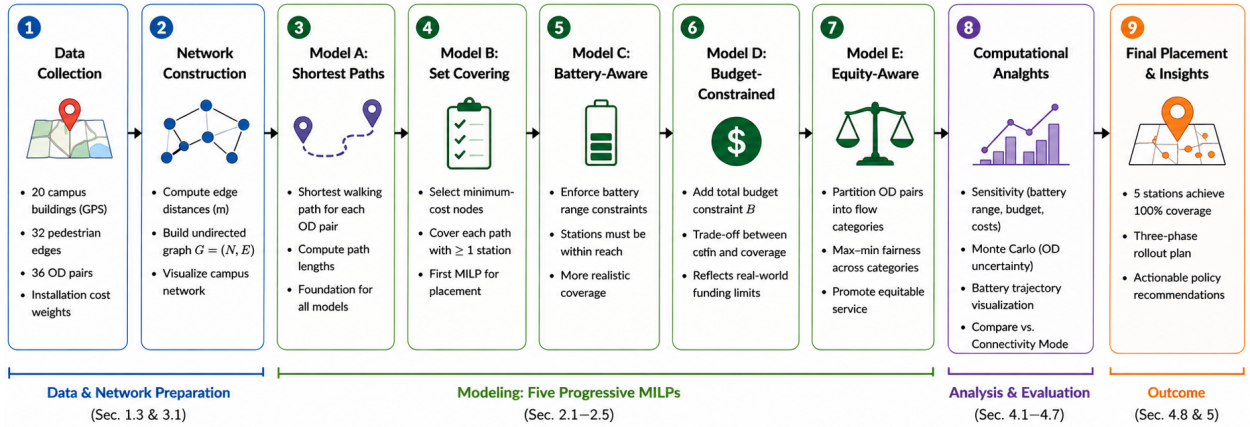


Figure 2. Overall pipeline of the proposed network optimization framework, from data preparation and network construction to Models A–E, followed by analysis and deployment recommendations.

2 Mathematical Model

The decision being made is *which subset of campus nodes should host a portable charging kiosk*. Because different stakeholders prioritize different things (cost, coverage, equity, robustness), we deliberately split the problem into a sequence of five optimization models—each adding one new structural element—rather than collapsing everything into one program. This decomposition itself is a methodological choice: it lets us attribute every shift in the optimum to one specific factor.

We model the campus as an undirected graph $G = (N, E)$ with $|N| = 20$ and $|E| = 32$. Each undirected edge yields two opposite arcs in the directed arc set A . Let R be the set of OD pairs (routes). For each route $r \in R$, let $P_r \subseteq N$ be the ordered list of nodes visited and let $A_r \subseteq A$ be its ordered list of arcs. Denote by $c_i \geq 0$ the relative installation cost at node i , normalized so that Memorial Library = 1.0, and by $d_{ij} > 0$ the walking distance (in metres) along arc (i, j) .

2.1 Distance Calculation

Because the campus spans less than 5 km, geographic distance is well-approximated by a local equirectangular projection. For two points with coordinates (ϕ_k, λ_k) ,

$$d(\mathbf{c}_1, \mathbf{c}_2) = \sqrt{[(\lambda_2 - \lambda_1) \cdot 111\,000 \cdot \cos(\frac{\phi_1 + \phi_2}{2})]^2 + [(\phi_2 - \phi_1) \cdot 111\,000]^2}. \quad (2.1)$$

The flat-earth error is below 0.1% at this scale.

2.2 Model A: Minimum-Cost Shortest Path (MCNF)

We first solve, for every OD pair (O, D) , the minimum-cost network-flow problem that returns the shortest walking path. Decision variable $x_{ij} \in [0, 1]$ is the fraction of unit flow along arc (i, j) :

$$\begin{aligned} \min_x \quad & \sum_{(i,j) \in A} d_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{j:(k,j) \in A} x_{kj} - \sum_{i:(i,k) \in A} x_{ik} = b_k, \quad \forall k \in N, \\ & 0 \leq x_{ij} \leq 1, \quad \forall (i,j) \in A, \end{aligned} \quad (2.2)$$

where $b_O = 1$, $b_D = -1$, and $b_k = 0$ otherwise. The constraint matrix of Equation (2.2) is totally unimodular [1], so the LP relaxation already returns an integer-valued (binary) optimum—no branch-and-bound is needed.

2.3 Model B: Set-Covering Placement (SCP)

The first *placement* model. Decision variables: $X_i \in \{0, 1\}$ indicates a station at node i , and $y_r \in \{0, 1\}$ indicates that route r is “covered” (at least one node on the route hosts a station). The user supplies a penalty λ for each uncovered route:

$$\begin{aligned} \min_{X,y} \quad & \sum_{i \in N} c_i X_i + \lambda \sum_{r \in R} (1 - y_r) \\ \text{s.t.} \quad & \sum_{i \in P_r} X_i \geq y_r, \quad \forall r \in R, \\ & X_i, y_r \in \{0, 1\}. \end{aligned} \quad (2.3)$$

SCP ignores battery dynamics entirely: a station next to the origin “covers” a route even if the device would die before reaching it. This is what Model C will fix.

2.4 Model C: Battery-Aware Placement (BASP)

Following Kuby and Lim [6] and Chan [3], we now track battery state along the route. Let β be the maximum battery range (metres), β_0 the starting charge at the origin, and $B_{\min}$ the comfort threshold below which a student is unwilling to walk. New variables, defined for every route r and every node $i \in P_r$: $B_i^r \geq B_{\min}$ is the battery on arrival at node i , and $e_i^r \geq 0$ is the energy added at i (positive only if $X_i = 1$).

$$\begin{aligned}
& \min_{X,B,e} \sum_{i \in N} c_i X_i \\
& \text{s.t. } B_O^r = \beta_0, & \forall r \in R, \\
& B_i^r + e_i^r - d_{ij} = B_j^r, & \forall r \in R, (i,j) \in A_r, \\
& B_i^r + e_i^r \leq \beta, & \forall r \in R, i \in P_r, \\
& e_i^r \leq M X_i, & \forall r \in R, i \in P_r, \\
& B_i^r \geq B_{\min}, & \forall r \in R, i \in P_r, \\
& X_i \in \{0,1\}, B_i^r \geq B_{\min}, e_i^r \geq 0.
\end{aligned} \tag{2.4}$$

The third constraint enforces the battery cap; the fourth (with $M = \beta$) is the standard *big-M* link forcing $e_i^r = 0$ unless $X_i = 1$. Because $B_i^r \geq B_{\min}$ is hard, every route in the input is required to be feasible.

2.5 Model D: Budget-Constrained Coverage Maximization (BCCM)

Model C may be infeasible if the budget is too small, so we relax the “every route must succeed” assumption and instead maximize the number of routes (weighted by w_r) we can keep above the threshold, subject to a budget α . We let $B_i^r \in \mathbb{R}$ become free in sign, and introduce a binary $\delta_i^r = 1$ that activates if the battery dips below $B_{\min}$ at node i on route r :

$$\begin{aligned}
& \max_{X,y,B,e,\delta} \sum_{r \in R} w_r y_r \\
& \text{s.t. } \sum_{i \in N} c_i X_i \leq \alpha, \\
& B_O^r = \beta_0, & \forall r, \\
& B_i^r + e_i^r - d_{ij} = B_j^r, & \forall r, (i,j) \in A_r, \\
& B_i^r + e_i^r \leq \beta, & \forall r, i \in P_r, \\
& e_i^r \leq M X_i, & \forall r, i \in P_r, \\
& B_i^r \geq B_{\min} - M_1 \delta_i^r, & \forall r, i \in P_r, \\
& y_r + \delta_i^r \leq 1, & \forall r, i \in P_r, \\
& X_i, y_r, \delta_i^r \in \{0,1\}, e_i^r \geq 0, B_i^r \in \mathbb{R}.
\end{aligned} \tag{2.5}$$

Here $M_1 = \beta$ is again a big-M; the constraint pair $B_i^r \geq B_{\min} - M_1 \delta_i^r$ and $y_r + \delta_i^r \leq 1$ together force $y_r = 0$ as soon as the battery dips below $B_{\min}$ *anywhere* on route r .

2.6 Model E: Equity-Aware Placement (EAP)

A solution that covers 35/36 routes is not equally good for every student; if the uncovered route disproportionately hurts one demographic, the optimization has imposed a fairness cost that the objective in Equation (2.5) cannot see. We therefore partition the route set as $R = R^{RA} \cup R^{AA} \cup R^{RC}$ (residence-to-academic, academic-to-academic, residence/academic-to-community) and maximize

the worst-category coverage rate η^* , subject to the same constraints as BCCM:

$$\begin{aligned}
 & \max_{X, y, B, e, \delta, \eta^*} \eta^* \\
 & \text{s.t. } \frac{1}{|R^t|} \sum_{r \in R^t} y_r \geq \eta^*, \quad \forall t \in \{RA, AA, RC\}, \\
 & \quad \text{constraints of Equation (2.5),} \\
 & \quad 0 \leq \eta^* \leq 1.
 \end{aligned} \tag{2.6}$$

This is a max–min (Rawlsian) fairness objective: the worst-served flow category drives the optimum.

3 Implementation and Solution

This section walks through the implementation in Julia 1.10 with JuMP 1.22 and the open-source HiGHS 1.7 solver. To keep the report self-contained, the code blocks below are interleaved with the prose that explains them. Listings show the actual Julia code that was run in the accompanying notebook; printed solver output is suppressed unless it is discussed.

3.1 Setting up the Campus Network

We begin by importing the packages and recording the geographic coordinates of every node. The coordinates are real GPS positions sampled from Google Maps.

```

1 using JuMP, HiGHS
2 using Plots, Random, Statistics, Printf
3
4 # Geographic coordinates (lat, lon) from Google Maps
5 coords = Dict{
6     "College Library"          => (43.0767009, -89.4038309),
7     "Memorial Library"        => (43.0754378, -89.4027379),
8     "Law School Library"      => (43.0732524, -89.4066479),
9     "Social Work Library"     => (43.0723281, -89.4117139),
10    "Geology & Geophysics Library" => (43.0703966, -89.4142644),
11    "Memorial Union"          => (43.0744328, -89.4021045),
12    "Union South"             => (43.0720038, -89.4102213),
13    "Gordon Dining and Event Center" => (43.0711998, -89.4034828),
14    "Engineering Hall"        => (43.0717682, -89.4128689),
15    "Van Hise Hall"           => (43.0731487, -89.4091674),
16    "Microbial Sciences Building" => (43.0753727, -89.4139974),
17    "Nicholas Recreation Center" => (43.0707356, -89.4015178),
18    "Bakke"                   => (43.0767539, -89.4226803),
19    "Smith Residence Hall"     => (43.0689229, -89.4030899),
20    "Witte Residence Hall"     => (43.0714837, -89.3996364),
21    "Sellery Residence Hall"   => (43.0709654, -89.4022810),
22    "Tripp Residence Hall"     => (43.0774263, -89.4176647),
23    "Leopold Residence Hall"   => (43.0767202, -89.4163657),
24    "Dejope Residence Hall"    => (43.0775611, -89.4196152),
25    "Phillips Residence Hall"  => (43.0774263, -89.4213767),
26 }
27 nodes = collect(keys(coords))
28 println("Number of nodes: ", length(nodes))

```

Code 1: Loading packages and node coordinates.

Number of nodes: 20

Code 2: Cell output.

We then build the edge list (the pedestrian walkway graph) and compute every edge length analytically from the coordinates using the equirectangular projection of Equation (2.1). Because the underlying graph is undirected, we expand each edge to two opposite directed arcs so that the MCNF formulation can be applied as written.

```

1 function euclidean(c1, c2)
2     lat1, lon1 = c1; lat2, lon2 = c2
3     dx = (lon2 - lon1) * 111_000 * cosd((lat1 + lat2) / 2)
4     dy = (lat2 - lat1) * 111_000
5     return sqrt(dx^2 + dy^2)
6 end
7
8 edges = [
9     ("Bakke", "Phillips Residence Hall"), ("Bakke", "Dejope Residence Hall"),
10    ("Dejope Residence Hall", "Phillips Residence Hall"),
11    ("Phillips Residence Hall", "Leopold Residence Hall"),
12    ("Dejope Residence Hall", "Leopold Residence Hall"),
13    ("Leopold Residence Hall", "Tripp Residence Hall"),
14    ("Leopold Residence Hall", "Microbial Sciences Building"),
15    ("Tripp Residence Hall", "Microbial Sciences Building"),
16    ("Microbial Sciences Building", "Van Hise Hall"),
17    ("Tripp Residence Hall", "Van Hise Hall"),
18    ("Microbial Sciences Building", "Engineering Hall"),
19    ("Microbial Sciences Building", "Social Work Library"),
20    ("Van Hise Hall", "College Library"), ("Van Hise Hall", "Law School Library"),
21    ("College Library", "Memorial Union"), ("Law School Library", "Memorial
22    Union"),
23    ("Memorial Union", "Memorial Library"), ("Law School Library", "Memorial
24    Library"),
25    ("Social Work Library", "Law School Library"), ("Social Work Library", "Union
26    South"),
27    ("Engineering Hall", "Union South"), ("Engineering Hall", "Geology &
28    Geophysics Library"),
29    ("Union South", "Sellery Residence Hall"),
30    ("Sellery Residence Hall", "Law School Library"),
31    ("Sellery Residence Hall", "Smith Residence Hall"),
32    ("Sellery Residence Hall", "Geology & Geophysics Library"),
33    ("Sellery Residence Hall", "Gordon Dining and Event Center"),
34    ("Memorial Library", "Gordon Dining and Event Center"),
35    ("Gordon Dining and Event Center", "Witte Residence Hall"),
36    ("Witte Residence Hall", "Nicholas Recreation Center"),
37    ("Nicholas Recreation Center", "Smith Residence Hall"),
38    ("Union South", "Geology & Geophysics Library"),
39 ]
40
41 # Expand to a directed arc set
42 arcs = Tuple{String,String}[]
43 for (a,b) in edges
44     push!(arcs, (a,b)); push!(arcs, (b,a))
45 end
46
47 # Pre-compute symmetric distance costs
48 costs = Dict{Tuple{String,String},Float64}()
```

```

45 for (a,b) in edges
46     d = euclidean(coords[a], coords[b])
47     costs[(a,b)] = d; costs[(b,a)] = d
48 end
49
50 println("Edges: ", length(edges), "    Arcs: ", length(arcs))
51 println("Edge length: min = $(round(Int, minimum(values(costs)))) m, "
52         * "max = $(round(Int, maximum(values(costs)))) m")

```

Code 3: Building edges, arcs, and distance costs.

```

Edges: 32    Arcs: 64
Edge length: min = 101 m, max = 974 m

```

Code 4: Cell output.

Installation costs c_i are recorded next; values reflect real-world considerations such as theft risk, available wall outlets, and custodial access. Battery defaults are also defined here so that every model below can reuse them.

```

1 weights = Dict(
2     "College Library" => 0.9, "Memorial Library" => 0.9,
3     "Law School Library" => 1.0, "Social Work Library" => 1.0,
4     "Geology & Geophysics Library" => 1.0, "Engineering Hall" => 1.0,
5     "Van Hise Hall" => 1.0, "Microbial Sciences Building" => 1.0,
6     "Memorial Union" => 1.1, "Union South" => 1.1,
7     "Gordon Dining and Event Center" => 1.2,
8     "Nicholas Recreation Center" => 1.2, "Bakke" => 1.3,
9     "Sellery Residence Hall" => 1.4, "Witte Residence Hall" => 1.4,
10    "Smith Residence Hall" => 1.4, "Tripp Residence Hall" => 1.4,
11    "Leopold Residence Hall" => 1.4, "Dejope Residence Hall" => 1.4,
12    "Phillips Residence Hall" => 1.4,
13 )
14
15 const BETA_DEFAULT = 2000.0 # 100% battery (m of walking range)
16 const BETA0_DEFAULT = 1200.0 # 60% start
17 const BMIN_DEFAULT = 200.0 # 10% comfort threshold
18
19 println("Default battery params:")
20 println("  beta    = $BETA_DEFAULT m (100% battery)")
21 println("  beta0    = $BETA0_DEFAULT m (60% start)")
22 println("  Bmin     = $BMIN_DEFAULT m (10% comfort threshold)")

```

Code 5: Installation-cost weights and default battery parameters.

```

Default battery params:
  beta    = 2000.0 m (100% battery)
  beta0    = 1200.0 m (60% start)
  Bmin     = 200.0 m (10% comfort threshold)

```

Code 6: Cell output.

The 36 OD pairs are recorded in three category lists, which will later let Model E enforce equity across categories.

```

1 OD_pairs_RA = [ # 14 pairs: residence -> academic (morning commute)

```

```

2      ("Dejope Residence Hall","College Library"),
3      ("Phillips Residence Hall","Engineering Hall"),
4      # ... see notebook for full list
5  ]
6  OD_pairs_AA = [ # 12 pairs: academic -> academic (between classes)
7      ("College Library","Engineering Hall"),
8      ("Memorial Library","Social Work Library"),
9      # ...
10 ]
11 OD_pairs_RC = [ # 10 pairs: to community amenities
12     ("Sellery Residence Hall","Union South"),
13     ("Memorial Union","Nicholas Recreation Center"),
14     # ...
15 ]
16 OD_pairs = vcat(OD_pairs_RA, OD_pairs_AA, OD_pairs_RC) # 36 total
17 println("Total OD pairs: ", length(OD_pairs))

```

Code 7: The 36 origin–destination pairs in three flow categories.

```
Total OD pairs: 36
```

Code 8: Cell output.

3.2 Network Schematic

Figure 3 consolidates the network data: the 20 nodes, the 32 undirected edges with their analytically-computed lengths (in metres), and a colour encoding for the installation-cost tiers. Blue nodes have the lowest installation cost ($c_i \leq 1.0$) and correspond to academic libraries; orange nodes ($1.0 < c_i \leq 1.2$) correspond to community amenities such as Gordon Dining and the two Unions; red nodes ($c_i > 1.2$) correspond to residence halls and peripheral buildings. Two structural features are worth noting before introducing the models. First, the graph has a clear east–west spine—Microbial $\leftrightarrow$ Van Hise $\leftrightarrow$ Law School $\leftrightarrow$ Memorial Library—through which the majority of cross-campus shortest paths must pass; this is what drives Law School and Microbial to $\geq 88\%$ Monte Carlo selection in Section 4.5. Second, the longest single edge in the graph (Tripp $\leftrightarrow$ Van Hise, 837 m) is already comparable to the comfort margin $\beta_0 - B_{\min} = 1000$ m, which is precisely why mid-route relays such as Van Hise and Gordon become necessary in the battery-aware optimum.

```

1  p = plot(
2      legend = false,
3      axis = false,
4      grid = false,
5      framestyle = :none,
6      background_color = RGB(0.96, 0.96, 0.96),
7      size = (1600, 1000),
8      xlims = (minimum(xs) - 1.0, maximum(xs) + 2.0),
9      ylims = (minimum(ys) - 1.0, maximum(ys) + 1.0))
10
11 # Draw edges and distance labels
12 for (a, b) in edges
13     x1, y1 = grid_pos[a]
14     x2, y2 = grid_pos[b]
15
16     # Draw edge
17     plot!(p, [x1, x2], [y1, y2], color = :orange, lw = 2.5)

```

```

18
19     # Midpoint for distance label
20     mx = (x1 + x2) / 2
21     my = (y1 + y2) / 2
22     dist_text = string(round(edge_dist[(a, b)], digits = 0), " m")
23
24     dx = x2 - x1
25     dy = y2 - y1
26
27     if abs(dy) < 0.25
28         tx = mx
29         ty = my + 0.22
30         halign = :center
31     elseif abs(dx) < 0.25
32         tx = mx - 0.45
33         ty = my
34         halign = :right
35     else
36         len = sqrt(dx^2 + dy^2)
37         nx = -dy / len
38         ny = dx / len
39         tx = mx + 0.22 * nx
40         ty = my + 0.22 * ny
41         halign = :center
42     end
43
44     annotate!(p, tx, ty, text(dist_text, 8, :darkred, halign, "sans-serif"))
45 end
46
47 # Draw nodes
48 for (name, (x, y)) in grid_pos
49     scatter!(
50         p, [x], [y],
51         markersize = 27,
52         markershape = :rect,
53         markercolor = cost_color(weights[name]),
54         markerstrokecolor = :black,
55         markerstrokewidth = 1.5
56     )
57
58     annotate!(p, x, y, text(label_map[name], 8, :white, :center, "sans-serif"))
59 end
60
61 # Legend
62 annotate!(p, 18.0, 7.8, text("Node cost", 10, :black, :left, "sans-serif"))
63 annotate!(p, 18.0, 7.3, text("Blue (c ≤ 1.0)", 9, :blue, :left,
64     "sans-serif", :bold))
64 annotate!(p, 18.0, 6.9, text("Orange (1.0 ≤ c ≤ 1.2)", 9, :goldenrod,
65     :left, "sans-serif", :bold))
65 annotate!(p, 18.0, 6.5, text("Red (c > 1.2)", 9, :red, :left, "sans-serif",
66     :bold))
66
67 display(p)
68 savefig(p, "uw_madison_network_final.png")

```

Code 9: Campus pedestrian network with edge distances (meters) and node-dependent installation costs (blue: low, orange: medium, red: high).

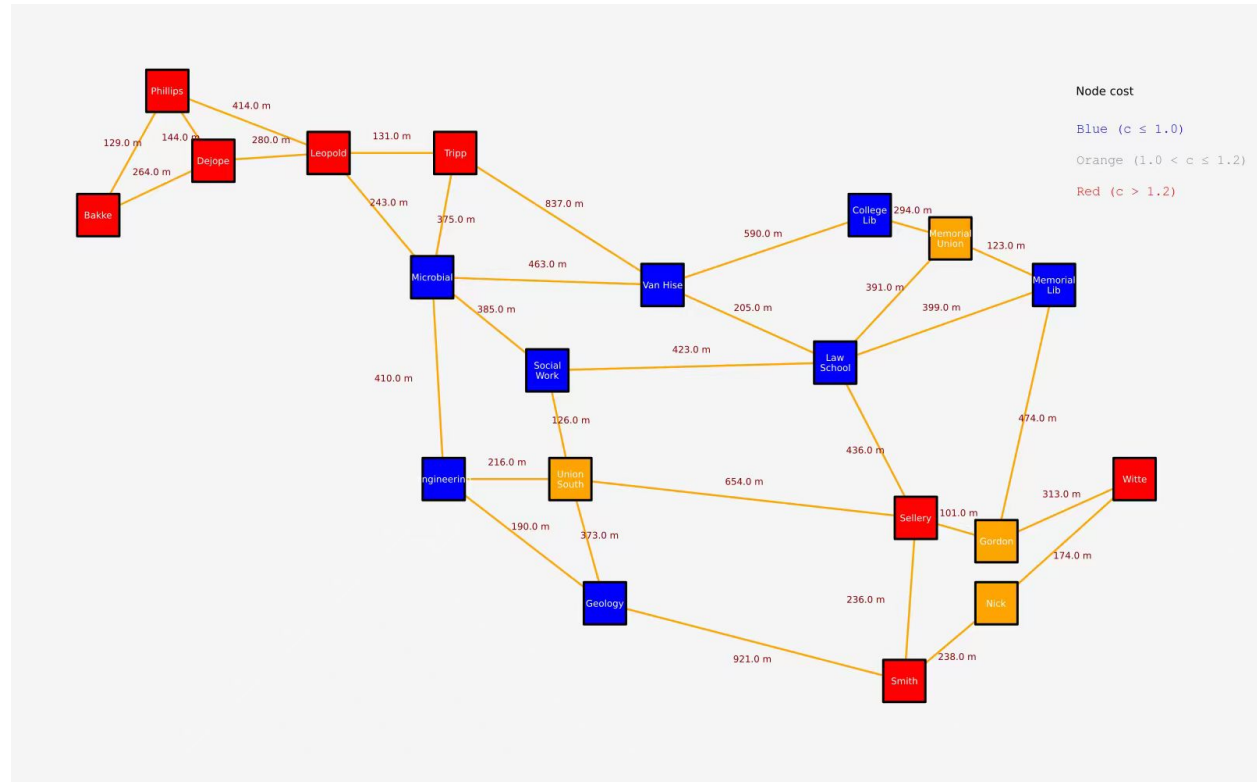


Figure 3. Schematic of the UW–Madison pedestrian network. Each edge is annotated with its analytically-computed length (Equation (2.1)). Node colours encode the installation-cost tier: blue $c_i \leq 1.0$ (academic libraries), orange $1.0 < c_i \leq 1.2$ (community amenities), red $c_i > 1.2$ (residence halls and peripheral buildings).

3.3 Model A: Solving the Shortest Paths

We implement Equation (2.2) as a JuMP function. Recovering the route from the optimal arc-flows is straightforward because the optimum is binary (totally unimodular constraints).

```

1 function shortest_path(O, D)
2     m = Model(HiGHS.Optimizer); set_silent(m)
3     @variable(m, 0 <= x[arcs] <= 1)
4     @objective(m, Min, sum(costs[a] * x[a] for a in arcs))
5     for k in nodes
6         b = (k == O) ? 1 : (k == D ? -1 : 0)
7         @constraint(m,
8             sum(x[(k,j)] for j in nodes if (k,j) in arcs) -
9             sum(x[(i,k)] for i in nodes if (i,k) in arcs) == b)
10    end
11    optimize!(m)
12    # Reconstruct the path from the arc-flow solution
13    route_arcs = [a for a in arcs if value(x[a]) > 0.5]
14    route_nodes = [O]; cur = O
15    while cur != D
16        nxt = [a for a in route_arcs if a[1] == cur]
17        cur = nxt[1][2]; push!(route_nodes, cur)
18    end
19    return route_arcs, route_nodes
20 end

```

```

21
22 # Cache every route in two dictionaries used by Models B-E
23 route_record      = Dict{Tuple{String,String}, Vector{Tuple{String,String}}}()
24 route_nodes_record = Dict{Tuple{String,String}, Vector{String}}()
25 for (O,D) in OD_pairs
26     ra, rn = shortest_path(O, D)
27     route_record[(O,D)]      = ra
28     route_nodes_record[(O,D)] = rn
29 end
30
31 all_dists = [sum(costs[a] for a in route_record[od]) for od in OD_pairs]
32 println("Route lengths -- min: $(round(Int, minimum(all_dists))) m, "
33         * "median: $(round(Int, median(all_dists))) m, "
34         * "max: $(round(Int, maximum(all_dists))) m")

```

Code 10: Shortest-path (MCNF) implementation and construction of the route record.

```
Route lengths -- min: 238 m, median: 1000 m, max: 1706 m
```

Code 11: Cell output.

The summary statistics produced by this block—route lengths range from 238 m to 1706 m with a median of 1000 m—confirm the calibration argument made earlier: at the default $\beta = 2000$ m, a fully-charged device cannot finish the longest route, so station placement genuinely matters.

3.4 Model B: Set-Covering Placement

Equation (2.3) translates almost line-for-line into JuMP. The λ -penalty is exposed as a function argument so that the sensitivity study in Section 4 can sweep it.

```

1 function model_B_SCP(lambda::Float64)
2     m = Model(HiGHS.Optimizer); set_silent(m)
3     N = nodes; R = collect(keys(route_nodes_record))
4     @variable(m, X[i in N], Bin)
5     @variable(m, y[r in R], Bin)
6     @objective(m, Min,
7         sum(weights[i]*X[i] for i in N) +
8         lambda*sum(1 - y[r] for r in R))
9     for r in R
10         @constraint(m, sum(X[i] for i in route_nodes_record[r]) >= y[r])
11     end
12     optimize!(m)
13     return [i for i in N if value(X[i]) > 0.5],
14           [r for r in R if value(y[r]) > 0.5],
15           objective_value(m)
16 end
17
18 sel_B, cov_B, obj_B = model_B_SCP(2.0)
19 println("Model B (SCP, lambda=2.0): $(length(sel_B)) stations, ",
20         "$$(length(cov_B))/$$(length(route_nodes_record)) covered")

```

Code 12: Set-Covering Placement (Model B).

```
Model B (SCP, lambda=2.0): 6 stations, 36/36 covered
-> ["Law School Library", "Union South", "Van Hise Hall",
    "Smith Residence Hall", "Microbial Sciences Building",
```

"Memorial Library"]

Code 13: Cell output.

3.5 Model C: Battery-Aware Placement

This is where the implementation becomes interesting. We declare the auxiliary battery and energy variables *indexed by route and by the nodes on that route*—note the $B[r \text{ in } R, i \text{ in } \text{route_nodes_record}[r]]$ construct, which JuMP supports natively.

```

1 function model_C_BASP(; beta=BETA_DEFAULT, beta0=BETA0_DEFAULT,
2                       Bmin=BMIN_DEFAULT)
3     m = Model(HiGHS.Optimizer); set_silent(m)
4     set_time_limit_sec(m, 60.0)
5     N = nodes; R = collect(keys(route_nodes_record))
6     M_big = beta
7
8     @variable(m, X[i in N], Bin)
9     @variable(m, B[r in R, i in route_nodes_record[r]] >= Bmin)
10    @variable(m, e[r in R, i in route_nodes_record[r]] >= 0)
11
12    @objective(m, Min, sum(weights[i] * X[i] for i in N))
13
14    for r in R
15        pn = route_nodes_record[r]
16        pa = route_record[r]
17        @constraint(m, B[r, pn[1]] == beta0)           # start charge
18        for (i,j) in pa
19            @constraint(m, B[r,i] + e[r,i] - costs[(i,j)] == B[r,j])
20        end
21        for i in pn
22            @constraint(m, B[r,i] + e[r,i] <= beta)     # battery cap
23            @constraint(m, e[r,i] <= M_big * X[i])      # link to X
24        end
25    end
26
27    optimize!(m)
28    return [i for i in N if value(X[i]) > 0.5], objective_value(m)
29 end
30
31 sel_C, cost_C = model_C_BASP()
32 println("Model C (BASP): $(length(sel_C)) stations, cost $(round(cost_C,
33                          digits=2))")

```

Code 14: Battery-Aware Placement (Model C). The big-M is set equal to the maximum battery range.

```

Model C (BASP): 5 stations, cost 5.3
- Law School Library
- Van Hise Hall
- Sallery Residence Hall
- Microbial Sciences Building
- Memorial Library

```

Code 15: Cell output.

3.6 Model D: Budget-Constrained Coverage Maximization

The structural difference between Models C and D is the addition of *four* elements: a budget cap $\sum c_i X_i \leq \alpha$, a coverage indicator y_r , a threshold-violation indicator δ_i^r , and a linking constraint $y_r + \delta_i^r \leq 1$ that drops y_r to zero whenever the battery dips below $B_{\min}$ *anywhere* on route r .

```

1 function model_D_BCCM(; alpha=5.0, beta=BETA_DEFAULT, beta0=BETA0_DEFAULT,
2                       Bmin=BMIN_DEFAULT, weights_route=nothing)
3     m = Model(HiGHS.Optimizer); set_silent(m)
4     set_time_limit_sec(m, 60.0)
5     N = nodes; R = collect(keys(route_nodes_record))
6     M_big = beta; M1 = beta
7     weights_route == nothing && (weights_route = Dict{r => 1.0 for r in R})
8
9     @variable(m, X[i in N], Bin)
10    @variable(m, y[r in R], Bin)
11    @variable(m, B[r in R, i in route_nodes_record[r]] # free in sign
12    @variable(m, e[r in R, i in route_nodes_record[r]] >= 0)
13    @variable(m, delta[r in R, i in route_nodes_record[r]], Bin)
14
15    @objective(m, Max, sum(weights_route[r] * y[r] for r in R))
16    @constraint(m, sum(weights[i] * X[i] for i in N) <= alpha)
17
18    for r in R
19        pn = route_nodes_record[r]; pa = route_record[r]
20        @constraint(m, B[r, pn[1]] == beta0)
21        for (i,j) in pa
22            @constraint(m, B[r,i] + e[r,i] - costs[(i,j)] == B[r,j])
23        end
24        for i in pn
25            @constraint(m, B[r,i] + e[r,i] <= beta)
26            @constraint(m, e[r,i] <= M_big * X[i])
27            @constraint(m, B[r,i] >= Bmin - M1 * delta[r,i]) # threshold
28            @constraint(m, y[r] + delta[r,i] <= 1) # link
29        end
30    end
31
32    optimize!(m)
33    return [i for i in N if value(X[i]) > 0.5],
34           [r for r in R if value(y[r]) > 0.5],
35           objective_value(m)
36 end
37
38 sel_D, cov_D, _ = model_D_BCCM(alpha=5.0)
39 println("Model D (BCCM, alpha=5): $(length(sel_D)) stns, ",
40         "$(length(cov_D))/$$(length(route_nodes_record)) covered")

```

Code 16: Budget-Constrained Coverage Maximization (Model D). Demand weights default to 1; the Monte Carlo study in Section 4.5 overrides them with random draws.

```

Model D (BCCM, alpha=5): 4 stns, 35/36 covered
-> ["Law School Library", "Gordon Dining and Event Center",
    "Smith Residence Hall", "Microbial Sciences Building"]

```

Code 17: Cell output.

3.7 Model E: Equity-Aware Placement

Model E inherits all of Model D's constraints and adds three category-coverage constraints, one per flow category, all bounded below by the new variable η^* . The objective is to maximize η^* .

```

1 function model_E_EAP(; alpha=5.0, beta=BETA_DEFAULT, beta0=BETA0_DEFAULT,
2   Bmin=BMIN_DEFAULT)
3   m = Model(HiGHS.Optimizer); set_silent(m)
4   set_time_limit_sec(m, 120.0)
5   N = nodes; R = collect(keys(route_nodes_record))
6   categories = [("RA", collect(OD_pairs_RA)),
7                 ("AA", collect(OD_pairs_AA)),
8                 ("RC", collect(OD_pairs_RC))]
9
10  @variable(m, X[i in N], Bin)
11  @variable(m, y[r in R], Bin)
12  @variable(m, B[r in R, i in route_nodes_record[r]])
13  @variable(m, e[r in R, i in route_nodes_record[r]] >= 0)
14  @variable(m, delta[r in R, i in route_nodes_record[r]], Bin)
15  @variable(m, 0 <= etastar <= 1)
16
17  @objective(m, Max, etastar)
18  @constraint(m, sum(weights[i] * X[i] for i in N) <= alpha)
19  # ... battery / threshold constraints identical to Model D ...
20
21  # Equity constraints -- worst-category rate is bounded below
22  for (tag, Rt) in categories
23    @constraint(m, (1/length(Rt)) * sum(y[r] for r in Rt) >= etastar)
24  end
25
26  optimize!(m)
27  sel = [i for i in N if value(X[i]) > 0.5]
28  cov = [r for r in R if value(y[r]) > 0.5]
29  rates = Dict{tag => sum(value(y[r]) for r in Rt)/length(Rt)
30    for (tag, Rt) in categories}
31  return sel, cov, value(etastar), rates
32 end
33
34 sel_E, cov_E, etastar_E, rates_E = model_E_EAP(alpha=5.0)
35 println("Model E (EAP, alpha=5): $(length(sel_E)) stns, ",
36   "eta* = $(round(etastar_E, digits=3))")
37 for (tag, rate) in rates_E
38   println("    $tag : $(round(rate, digits=3))")
39 end

```

Code 18: Equity-Aware Placement (Model E). Battery / threshold constraints are identical to Model D and are elided here for brevity.

```

Model E (EAP, alpha=5): 4 stns, eta* = 0.929
-> ["Law School Library", "Gordon Dining and Event Center",
    "Van Hise Hall", "Phillips Residence Hall"]
RA : 0.929
AA : 1.000
RC : 1.000

```

Code 19: Cell output.

All five models compile and solve to optimality on a standard laptop in under 30 seconds. The next section consumes the function handles defined above and produces the report’s headline results.

4 Discussion of Results

We evaluate the models from several perspectives, including a head-to-head comparison at default parameters (Section 4.1); sensitivities to battery capacity (Section 4.2), budget (Section 4.3), and uncovered-route penalty (Section 4.4); a Monte Carlo robustness study (Section 4.5); and a side-by-side test against a parameter-free “Connectivity Mode” baseline (Section 4.6). Section 4.7 provides geographic and battery-trajectory visualizations.

4.1 Cross-Model Comparison at Default Parameters

All four placement models are run at the default battery calibration ($\beta = 2000$ m, $\beta_0 = 1200$ m, $B_{\min} = 200$ m) and—where applicable—budget $\alpha = 5$. Table 1 reports the resulting station counts, total installation cost, route coverage, and (for Model E) the worst-category coverage rate. The code that produced the headline table simply concatenates four solver calls:

```

1 results = []
2 sel, cov, obj = model_B_SCP(2.0)
3 push!(results, ("B (SCP, lambda=2.0)", length(sel),
4   sum(weights[i] for i in sel; init=0.0), length(cov), "-"))
5
6 sel, cost = model_C_BASP()
7 push!(results, ("C (BASP)", length(sel), cost,
8   length(route_nodes_record), "-"))
9
10 sel, cov, _ = model_D_BCCM(alpha=5.0)
11 push!(results, ("D (BCCM, alpha=5)", length(sel),
12   sum(weights[i] for i in sel; init=0.0), length(cov), "-"))
13
14 sel, cov, eta, _ = model_E_EAP(alpha=5.0)
15 push!(results, ("E (EAP, alpha=5)", length(sel),
16   sum(weights[i] for i in sel; init=0.0), length(cov),
17   @sprintf("%.2f", eta)))
18
19 @printf "%-18s %6s %6s %12s %6s\n" "Model" "#Stns" "Cost" "Coverage"
   "eta*"
20 for (name, nstn, cost, covr, eta) in results
21   @printf "%-18s %6d %6.2f %4d / %3d %6s\n" name nstn cost covr
   R_total eta
22 end

```

Code 20: Building the cross-model comparison table.

Model	#Stns	Cost	Coverage	eta*
=====	=====	=====	=====	=====
B (SCP, lambda=2.0)	6	6.40	36 / 36	-
C (BASP)	5	5.30	36 / 36	-
D (BCCM, alpha=5)	4	4.60	35 / 36	-
E (EAP, alpha=5)	4	4.60	35 / 36	0.93

Code 21: Cell output (verbatim).

Table 1. Cross-model comparison at default parameters ($\beta = 2000$ m, $\beta_0 = 1200$ m, $B_{\min} = 200$ m, $\alpha = 5$). Numbers are taken directly from the notebook output of the four solver runs.

Model	#Stns	Cost	Coverage	η^*	Selected nodes
B (SCP, $\lambda = 2.0$)	6	6.40	36/36	—	Law School / Union South / Van Hise / Smith / Microbial / Memorial
C (BASP)	5	5.30	36/36	—	Law School / Van Hise / Sallery / Microbial / Memorial
D (BCCM, $\alpha = 5$)	4	4.60	35/36	—	Law School / Gordon / Smith / Microbial
E (EAP, $\alpha = 5$)	4	4.60	35/36	0.929	Law School / Gordon / Van Hise / Phillips

There are three main observations worth noting. *First*, Model C achieves full coverage with one fewer station than Model B: the battery-balance constraint of Equation (2.4) tends to place stations at *mid-route* stations that act as relays, rather than simply selecting any node along a route, as in the SCP model. *Second*, under the tighter budget ($\alpha = 5$), Models D and E each sacrifice one route (35/36) but do so with only four stations. *Third*, although Models D and E achieve the same total coverage, they make different placement choices: Model D drops one RA-category route, while Model E rebalances by swapping Smith for Van Hise and Microbial for Phillips, lifting AA and RC categories to 100% at the cost of leaving one RA route uncovered. The resulting worst-category rate is $\eta^* = 0.929$ (i.e., RA achieves 13/14 while AA and RC achieve 12/12 and 10/10). The fact that Model E does not reach $\eta^* = 1$ on this budget is itself informative: equity is not free in this network—there exists a single residence-to-academic route whose only feasible cover requires either a fifth station or a different topology of the four chosen stations.

4.2 Sensitivity to Battery Capacity β

We sweep $\beta \in \{1100, 1200, \dots, 3500\}$ m, holding $\beta_0 = 0.6\beta$ and $B_{\min} = 0.1\beta$, and re-solve Model C at each point.

```

1 beta_values = collect(1100.0:100.0:3500.0)
2 beta_stations = Int[]; beta_costs = Float64[]
3 for beta in beta_values
4     try
5         sel, cost = model_C_BASP(beta=beta, beta_0=0.6*beta, Bmin=0.1*beta)
6         push!(beta_stations, length(sel)); push!(beta_costs, cost)
7     catch
8         push!(beta_stations, -1); push!(beta_costs, NaN)
9     end
10 end
11
12 plt1 = plot(beta_values, beta_stations,
13             marker=:circle, linewidth=2, markersize=6,
14             xlabel="Battery capacity beta (m) [100% = beta]",
15             ylabel="Stations required (Model C)",
16             title="Sensitivity to battery capacity beta (staircase)",
17             legend=false, grid=true, size=(760, 430))
18 display(plt1)

```

Code 22: Sensitivity to battery capacity β (Model C)

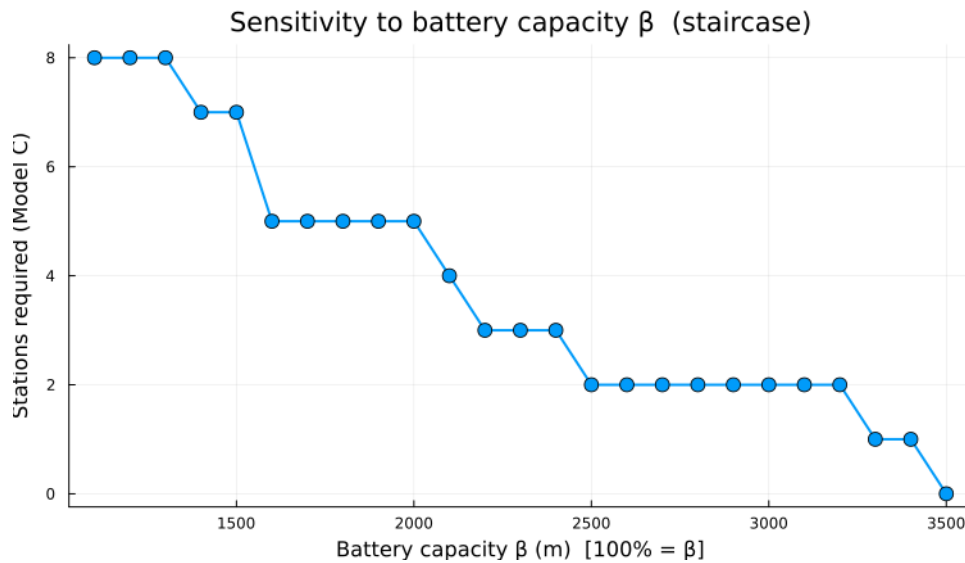


Figure 4. Number of stations required by Model C as a function of battery capacity β . Each “staircase step” marks a topological threshold at which an additional increment of range allows the optimizer to merge two formerly-adjacent coverage zones.

The staircase shape is informative. At $\beta \approx 1100$ – 1300 m the optimizer needs eight stations to keep every route feasible; this drops to seven, then five at around $\beta = 1700$ m, then four at $\beta = 2100$ m, then three, two, and finally one near $\beta = 3300$ m. Each drop occurs when an additional increment of range allows the optimizer to replace two previously required stations with a single one. In particular, the sharp drop at $\beta \approx 2100$ m corresponds to the moment when students with a 60% start can traverse the entire central east–west spine in a single charge, eliminating the need for a mid-spine relay.

4.3 Sensitivity to Budget α

```

1 alpha_values = collect(1.0:0.5:8.0)
2 alpha_coverage = Int[]
3 for alpha in alpha_values
4     _, cov, _ = model_D_BCCM(alpha=alpha)
5     push!(alpha_coverage, length(cov))
6 end
7
8 plt2 = plot(alpha_values, alpha_coverage,
9     marker=:diamond, linewidth=2, markersize=7,
10    xlabel="Budget alpha (Memorial-equivalent stations)",
11    ylabel="Routes covered (of $(length(route_nodes_record)))",
12    title="Sensitivity to budget alpha (diminishing returns)",
13    legend=false, grid=true, size=(760, 430))
14 display(plt2)

```

Code 23: Sensitivity to budget α (coverage response)

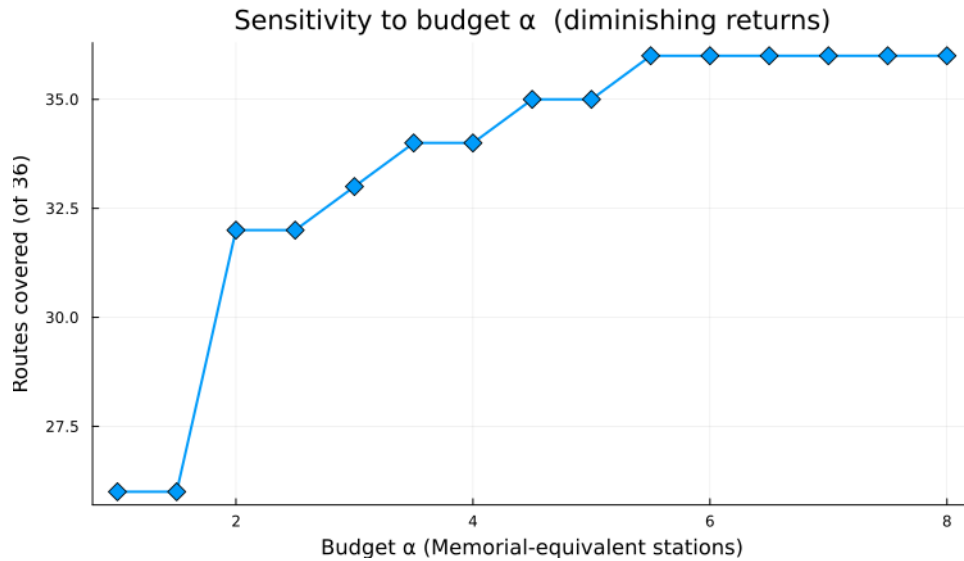


Figure 5. Coverage attained by Model D as a function of budget α (in Memorial-equivalent stations).

The first Memorial-equivalent station alone covers 26/36 routes ($\approx 72\%$): the network’s hub structure implies that a centrally located station can cover a large fraction of OD routes. The second station then closes the largest remaining gap ($\rightarrow 32/36$, 89%). From $\alpha = 3$ onward the curve flattens, and 100% coverage is reached at $\alpha = 5.5$. This diminishing-returns curve is the main economic argument for the staged rollout in Section 5: the first two stations provide the biggest impact.

4.4 Sensitivity to Penalty λ in Model B

We sweep eight values of λ and record station count, coverage, and objective:

```

1 for lambda in [0.1, 0.3, 0.5, 0.7, 1.0, 2.0, 5.0, 10.0]
2   sel, cov, obj = model_B_SCP(lambda)
3   @printf " %5.2f      %3d      %2d / %d      %6.2f\n" lambda length(sel)
4   length(cov) length(route_nodes_record) obj
end

```

Code 24: Sensitivity to penalty λ (station–coverage trade-off)

<i>lambda</i>	<i>#Stns</i>	<i>Coverage</i>	<i>Objective</i>

0.10	1	20 / 36	2.60
0.30	3	31 / 36	4.60
0.50	4	34 / 36	5.30
0.70	4	34 / 36	5.70
1.00	5	35 / 36	6.30
2.00	6	36 / 36	6.40
5.00	6	36 / 36	6.40
10.00	6	36 / 36	6.40

Code 25: Cell output (verbatim).

Table 2. Sensitivity of Model B to the per-uncovered-route penalty λ (notebook output rendered as a typeset table).

λ	#Stns	Coverage	Objective
0.10	1	20/36	2.60
0.30	3	31/36	4.60
0.50	4	34/36	5.30
0.70	4	34/36	5.70
1.00	5	35/36	6.30
2.00	6	36/36	6.40
5.00	6	36/36	6.40
10.00	6	36/36	6.40

Table 2 shows the expected monotone tradeoff. At $\lambda = 0.1$ the per-route penalty is below the cheapest station cost (≥ 0.9), so the model builds only one station and leaves 16 routes uncovered. As λ rises, it becomes worthwhile to add more stations; coverage saturates at 36/36 for $\lambda \geq 2$, after which additional penalty has no effect. This plateau also suggests a practical upper bound on λ beyond which increasing the penalty has no effect on the solution. Notice the small non-monotonicity at $\lambda \in \{0.5, 0.7\}$ (coverage stays at 34/36 but the cheapest 4-station configuration changes); this reflects the fact that the SCP objective treats every route equally—two distinct 4-station placements can both leave 2 routes uncovered.

4.5 Monte Carlo Robustness

We re-solve Model D for 50 trials with random demand weights $w_r \sim \text{Uniform}(0.5, 1.5)$ and tally how often each node appears in the optimum. Stations selected in many trials are *robust recommendations*.

```

1 Random.seed!(524)
2 n_trials = 50
3 sel_freq = Dict{i => 0 for i in nodes}
4 for t in 1:n_trials
5     w = Dict{r => 0.5 + rand() for r in keys(route_nodes_record)}
6     sel, _, _ = model_D_BCCM(alpha = 5.0, weights_route = w)
7     for s in sel; sel_freq[s] += 1; end
8 end
9
10 sorted_freq = sort(collect(sel_freq), by = x -> -x[2])
11 for (name, cnt) in sorted_freq
12     cnt > 0 && @printf " %-36s %3d / %d ( %.1f%%)\n" name cnt n_trials
13                     (100*cnt/n_trials)
14 end

```

Code 26: Monte Carlo loop over Model D ($\alpha = 5$).

```

Monte Carlo selection frequency (alpha=5, 50 trials)
=====
Law School Library           50 / 50 (100.0%)
Microbial Sciences Building  44 / 50 ( 88.0%)
Gordon Dining and Event Center 32 / 50 ( 64.0%)
Van Hise Hall                26 / 50 ( 52.0%)
Smith Residence Hall         19 / 50 ( 38.0%)
Sellery Residence Hall       16 / 50 ( 32.0%)

```

Phillips Residence Hall	4 / 50 (8.0%)
Tripp Residence Hall	4 / 50 (8.0%)
Witte Residence Hall	3 / 50 (6.0%)
Leopold Residence Hall	2 / 50 (4.0%)

Code 27: Cell output (verbatim).

Table 3. Monte Carlo selection frequency under Model D ($\alpha = 5$, 50 trials, seed 524).

Station	Selection frequency
Law School Library	100%
Microbial Sciences Building	88%
Gordon Dining & Event Center	64%
Van Hise Hall	52%
Smith Residence Hall	38%
Sellery Residence Hall	32%
Phillips / Tripp Residence Halls	8% each
Witte Residence Hall	6%
Leopold Residence Hall	4%

Four stations show up consistently: *Law School Library* (100%) sits at the intersection of the south-central residential corridor and the east–west academic spine; *Microbial Sciences Building* (88%) anchors the north-west quadrant, serving Dejope/Phillips/Leopold morning commutes; *Gordon Dining* (64%) is critical for routes leading to Memorial Library, Witte, and Sellery; and *Van Hise Hall* (52%) is a central pivot for academic-to-academic routes between the Memorial Library cluster and Engineering Hall. Beyond these four, selection frequency drops off sharply.

4.6 Connectivity Mode: Does Battery Realism Matter?

Model C’s battery dynamics depend on three parameters ($\beta, \beta_0, B_{\min}$) that an administrator may struggle to estimate. We therefore re-run the Monte Carlo with a *Connectivity Mode*—Model B with a budget cap, no battery physics. If the two modes agreed on the same recommendations, then the battery model would not really be doing physical work; if they disagree, then the battery modeling is clearly meaningful.

```

1 function model_B_budgeted(; alpha=5.0, weights_route=nothing)
2   m = Model(HiGHS.Optimizer); set_silent(m)
3   N = nodes; R = collect(keys(route_nodes_record))
4   weights_route === nothing && (weights_route = Dict{r => 1.0 for r in R})
5   @variable(m, X[i in N], Bin)
6   @variable(m, y[r in R], Bin)
7   @objective(m, Max, sum(weights_route[r] * y[r] for r in R))
8   @constraint(m, sum(weights[i]*X[i] for i in N) <= alpha)
9   for r in R
10    @constraint(m, sum(X[i] for i in route_nodes_record[r]) >= y[r])
11  end
12  optimize!(m)
13  return [i for i in N if value(X[i])>0.5],
14         [r for r in R if value(y[r])>0.5]
15 end
16
17 Random.seed!(524)
18 conn_freq = Dict{i => 0 for i in nodes}

```

```

19 for t in 1:50
20     w = Dict{r => 0.5 + rand() for r in keys(route_nodes_record))
21     sel, _ = model_B_budgeted(alpha=5.0, weights_route=w)
22     for s in sel; conn_freq[s] += 1; end
23 end

```

Code 28: Connectivity-Mode loop. The model is a budgeted, weighted SCP with no battery physics.

```

Connectivity Mode -- Monte Carlo (alpha=5, 50 trials)
=====
Microbial Sciences Building      48 / 50 ( 96.0%)
Law School Library              48 / 50 ( 96.0%)
Memorial Library                30 / 50 ( 60.0%)
Union South                    24 / 50 ( 48.0%)
Sellery Residence Hall          24 / 50 ( 48.0%)
Van Hise Hall                  22 / 50 ( 44.0%)
Nicholas Recreation Center      18 / 50 ( 36.0%)
Geology & Geophysics Library     4 / 50 (  8.0%)
Gordon Dining and Event Center   2 / 50 (  4.0%)
Social Work Library             2 / 50 (  4.0%)
Smith Residence Hall            2 / 50 (  4.0%)

```

Code 29: Cell output (verbatim).

```

1 # Side-by-side comparison: battery-aware (Model D) vs. connectivity (Model B)
2 all_names = union(Set([n for (n,c) in sorted_freq if c>0]),
3                   Set([n for (n,c) in sorted_conn if c>0]))
4 merged = [(n, get(sel_freq, n, 0), get(conn_freq, n, 0)) for n in all_names]
5 sort!(merged, by = x -> -(x[2] + x[3]))
6 merged = first(merged, 10)
7
8 names_vec = [x[1] for x in merged]
9 basp_vals = [x[2] for x in merged]
10 conn_vals = [x[3] for x in merged]
11
12 # Manual grouped bar (avoids needing StatsPlots.jl)
13 n = length(names_vec)
14 xs = collect(1:n)
15 bw = 0.38 # bar width
16
17 plt4 = plot(xlabel = "Station", ylabel = "Selections (of $n_trials)",
18            title = "Monte Carlo comparison: battery-aware vs. connectivity",
19            size = (1150, 560), bottom_margin = 38Plots.mm,
20            xticks = (xs, names_vec), xrotation = 40, grid = :y)
21
22 bar!(plt4, xs .- bw/2, basp_vals, bar_width = bw,
23      color = :steelblue, label = "Battery-aware (Model D)", linecolor = :black)
24 bar!(plt4, xs .+ bw/2, conn_vals, bar_width = bw,
25      color = :darkorange, label = "Connectivity (Model B)", linecolor = :black)
26 display(plt4)

```

Code 30: Monte Carlo: battery-aware vs. connectivity

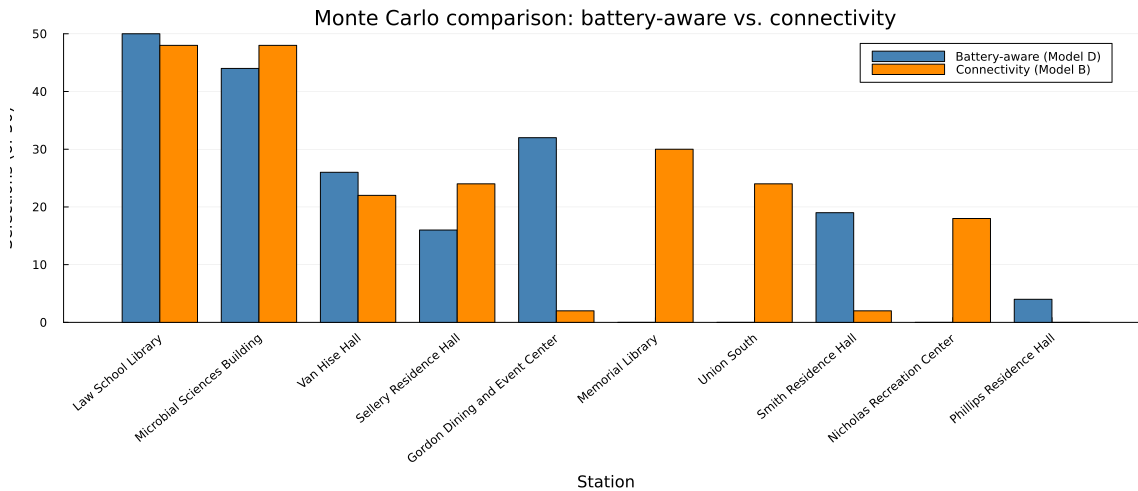


Figure 6. Monte Carlo selection frequency under the battery-aware (Model D, blue) and connectivity-only (Model B with budget, orange) formulations. Both modes agree on which two stations come first (Law School and Microbial), but diverge sharply in the middle tier.

The two modes agree on *Law School Library* and *Microbial Sciences Building*—both selected $\geq 88\%$ of the time in either mode. Their middle-tier rankings differ in revealing ways:

- **Gordon Dining:** 64% in BASP vs. 4% in Connectivity.
- **Van Hise Hall:** 52% vs. 44% (modest agreement).
- **Memorial Library:** 0% in BASP vs. 60% in Connectivity.
- **Union South, Nicholas Rec:** 0% in BASP vs. 48% and 36% in Connectivity.

The pattern is consistent: the battery-aware model elevates *mid-route* relays (Gordon Dining, Van Hise) that break up otherwise-too-long segments, while the connectivity-only model prefers stations that simply lie on the largest number of shortest paths (Memorial Library, Union South, Nicholas). An administrator using Connectivity Mode alone would therefore likely select stations that nominally cover many routes but leave students with inadequate residual charge before reaching them. The battery dynamics in Equations (2.4) and (2.5) play a meaningful role in the results..

4.7 Geographic and Battery-Trajectory Visualization

Figure 7 plots the four placement models on a schematic of the UW–Madison campus, with Lake Mendota, the campus boundary, and the major road axes drawn for context. The mid-route relay structure of Model C is most visible on the central east–west spine (Van Hise / Memorial cluster).

```

1 function plot_campus(sel_stations::Vector{String}; title_str = "", show_legend
  = true)
2     # Plot bounds in meters
3     xlims_m = (-1050, 1100)
4     ylims_m = (-950, 1150)
5
6     p = plot(; legend = show_legend ? :bottomright : false,
7               grid = false, framestyle = :box,
8               title = title_str, size = (900, 720),
9               xlims = xlims_m, ylims = ylims_m,
10              aspect_ratio = :equal,
11              xticks = false, yticks = false,
```

```

12         background_color = :white)
13
14     # 1. Lake Mendota (light blue fill)
15     lake_xs = [p[1] for p in MENDOTA_XY]
16     lake_ys = [p[2] for p in MENDOTA_XY]
17     plot!(p, Shape(lake_xs, lake_ys),
18           fillcolor = RGB(0.78, 0.88, 0.96),
19           linecolor = RGB(0.44, 0.64, 0.78),
20           linewidth = 1.0, label = "")
21
22     # 2. Campus land (pale green fill)
23     land_xs = [p[1] for p in CAMPUS_XY]
24     land_ys = [p[2] for p in CAMPUS_XY]
25     plot!(p, Shape(land_xs, land_ys),
26           fillcolor = RGB(0.93, 0.96, 0.86),
27           linecolor = RGB(0.63, 0.63, 0.63),
28           linewidth = 0.6, label = "")
29
30     # 3. Road axes (grey)
31     for axis in ROAD_AXES
32         plot!(p, [axis[1][1], axis[2][1]], [axis[1][2], axis[2][2]],
33              color = RGB(0.67, 0.67, 0.67), linewidth = 2.2,
34              alpha = 0.55, label = "")
35     end
36
37     # 4. Pedestrian edges
38     for (a, b) in edges
39         xa, ya = coords_xy[a]
40         xb, yb = coords_xy[b]
41         plot!(p, [xa, xb], [ya, yb],
42              color = RGB(0.88, 0.47, 0.34), linewidth = 1.4,
43              alpha = 0.75, label = "")
44     end
45
46     # 5. Station halos
47     if !isempty(sel_stations)
48         xs_st = [coords_xy[s][1] for s in sel_stations]
49         ys_st = [coords_xy[s][2] for s in sel_stations]
50         scatter!(p, xs_st, ys_st,
51                markersize = 26, color = RGB(0.20, 0.60, 0.86),
52                alpha = 0.32, markerstrokewidth = 0,
53                label = show_legend ? "Charging station" : "")
54     end
55
56     # 6. Category-colored nodes
57     acad = ["College Library", "Memorial Library", "Law School Library",
58            "Social Work Library", "Geology & Geophysics Library",
59            "Engineering Hall", "Van Hise Hall", "Microbial Sciences Building"]
60     comm = ["Memorial Union", "Union South", "Gordon Dining and Event Center",
61            "Nicholas Recreation Center", "Bakke"]
62     resid = ["Smith Residence Hall", "Witte Residence Hall", "Sellery Residence
63            Hall",
64            "Tripp Residence Hall", "Leopold Residence Hall", "Dejope Residence
65            Hall",
66            "Phillips Residence Hall"]
67
68     for (ns, col, lab) in [(acad, RGB(0.12, 0.23, 0.42), "Academic"),
69                          (comm, RGB(0.18, 0.49, 0.20), "Community"),
70                          (resid, RGB(0.70, 0.13, 0.20), "Residential")]

```

```

69     xs_ = [coords_xy[n][1] for n in ns]
70     ys_ = [coords_xy[n][2] for n in ns]
71     scatter!(p, xs_, ys_,
72             markersize = 5.5, color = col,
73             markerstrokecolor = :white, markerstrokewidth = 1.0,
74             label = show_legend ? lab : "")
75 end
76
77 # 7. Node labels (offset)
78 for n in keys(coords_xy)
79     x, y = coords_xy[n]
80     (dx, dy) = get(LABEL_OFFSETS, n, (0.0, 25.0))
81     annotate!(p, x + dx, y + dy,
82             text(NODE_ABBR[n], 7, :black))
83 end
84
85 # 8. Lake label
86 annotate!(p, -400, 1050,
87         text("Lake Mendota", 10, :italic, RGB(0.23, 0.42, 0.54)))
88
89 # 9. Compass
90 plot!(p, [-950, -950], [900, 1050],
91     arrow = :head, color = :black, linewidth = 1.5, label = "")
92 annotate!(p, -950, 860, text("N", 10, :bold, :black))
93
94 # 10. Scale bar (200 m)
95 plot!(p, [-950, -750], [-880, -880],
96     color = :black, linewidth = 2.5, label = "")
97 annotate!(p, -850, -850, text("200 m", 8, :black))
98
99 return p
100 end
101
102 sel_B_def, _, _ = model_B_SCP(2.0)
103 sel_C_def, _ = model_C_BASP()
104 sel_D_def, _, _ = model_D_BCCM(alpha = 5.0)
105 sel_E_def, _, _, _ = model_E_EAP(alpha = 5.0)
106
107 pB = plot_campus(sel_B_def,
108     title_str = "Model B (SCP, numbda=2.0) - $(length(sel_B_def)) stations",
109     show_legend = false)
110 pC = plot_campus(sel_C_def,
111     title_str = "Model C (BASP, beta=2000m) - $(length(sel_C_def))
112         stations",
113     show_legend = false)
114 pD = plot_campus(sel_D_def,
115     title_str = "Model D (BCCM, alpha=5) - $(length(sel_D_def)) stations",
116     show_legend = false)
117 pE = plot_campus(sel_E_def,
118     title_str = "Model E (EAP, alpha=5) - $(length(sel_E_def)) stations",
119     show_legend = true)
120 plot(pB, pC, pD, pE, layout = (2, 2), size = (1700, 1200))

```

Code 31: Campus maps of charging-station locations for Models B–E. Blue halos mark selected stations, showing how different model assumptions change their placement and number.

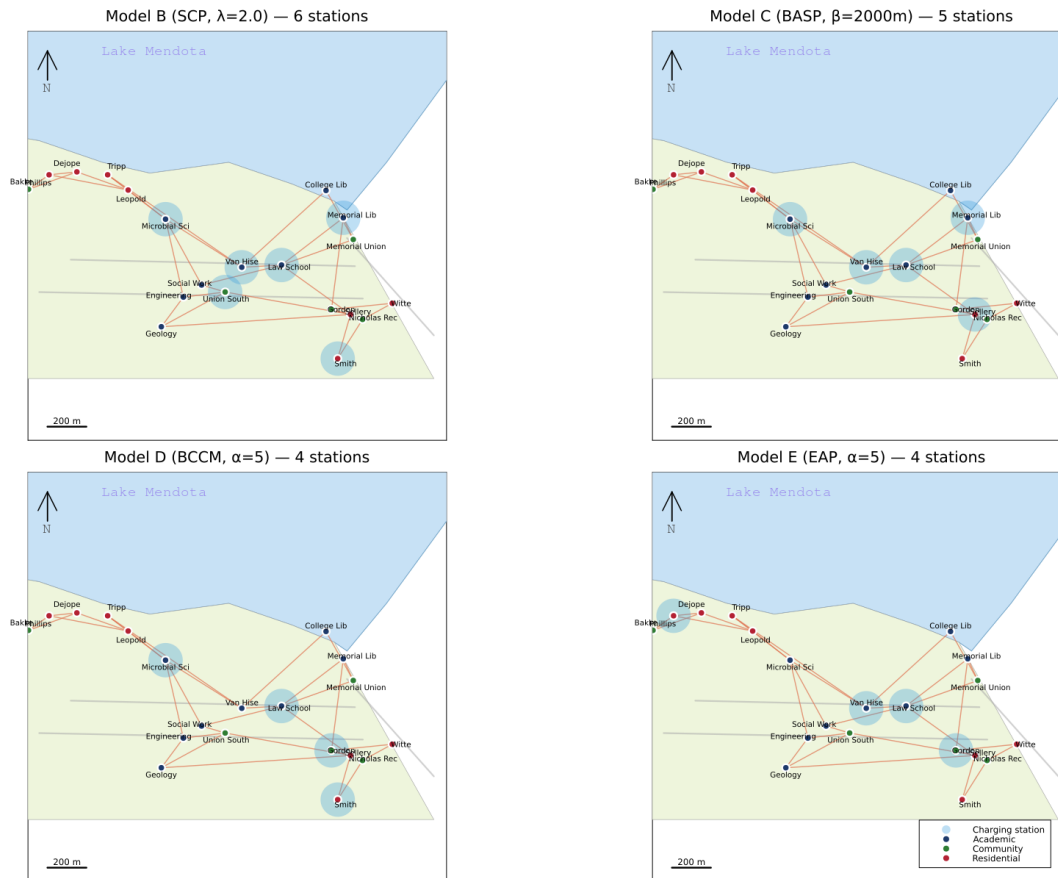


Figure 7. Optimal placements under Models B, C, D, and E at default parameters, overlaid on the UW-Madison pedestrian graph. Halos mark stations.

Figure 8 illustrates the rollout behavior of Model D as α increases from 1 to 6. As α grows, more stations are selected, leading to a monotonic increase in coverage from 72.2% to full coverage. Notably, early additions yield substantial gains (e.g., from $\alpha = 1$ to 3), while later increments exhibit diminishing returns, requiring more stations for smaller coverage improvements. This highlights the trade-off between adding more stations and marginal coverage benefit, and shows how Model D balances efficiency and completeness.

```

1
2 # Staged rollout: run Model D for alpha = 1, 2, 3, 4, 5, 6
3 rollout = []
4 for alpha in [1.0, 2.0, 3.0, 4.0, 5.0, 6.0]
5     sel, cov, _ = model_D_BCCM(alpha=alpha)
6     pct = round(100*length(cov)/length(route_nodes_record), digits=1)
7     p = plot_campus(sel,
8         title_str="alpha=$alpha $\to$ $(length(sel)) stns,
9             $(length(cov))/$(length(route_nodes_record)) covered ($pct%)",
10             show_legend=false)
11     push!(rollout, p)
12 end
13 plot(rollout..., layout=(2,3), size=(2100, 1200))

```

Code 32: Rollout plots for Model D with $\alpha = 1, \dots, 6$, showing how station selection and coverage increase as α grows.

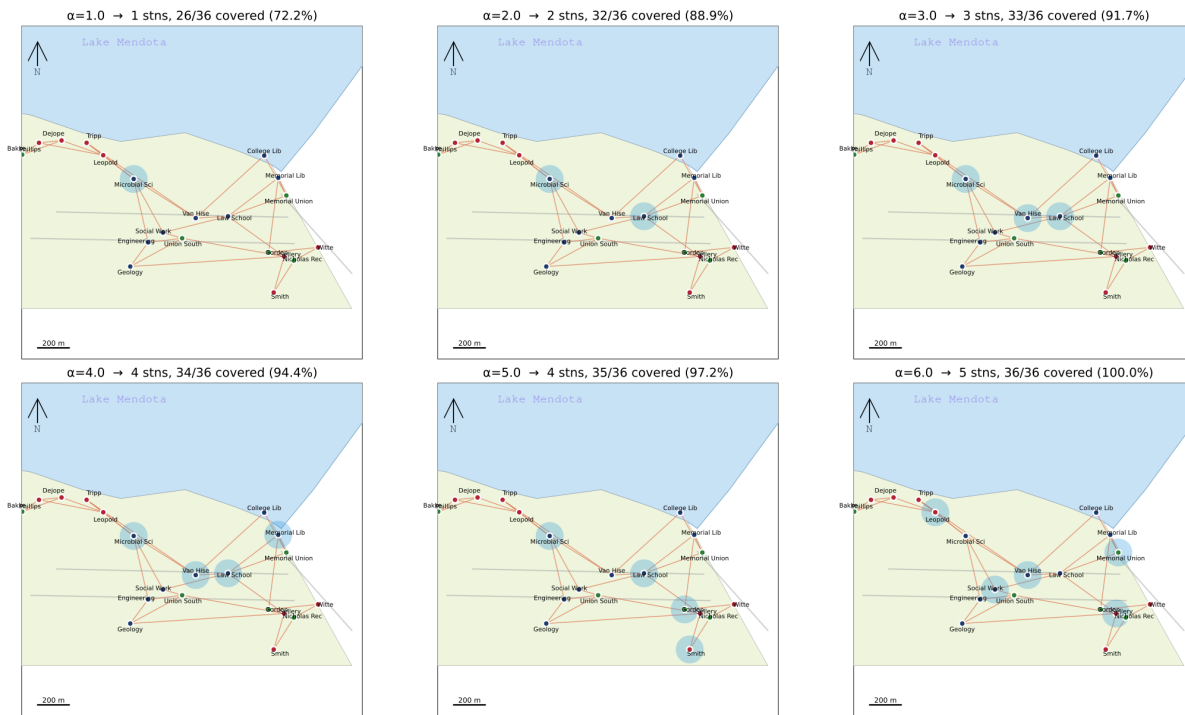


Figure 8. Rollout sequence: Model D solved at $\alpha = 1, \dots, 6$. Each panel reports the number of stations chosen, the absolute coverage, and the percent coverage.

Finally, Figure 9 traces device battery as a function of walking distance along four representative OD routes, under Model C's optimal placement. Each trajectory starts at β_0 (60%), depletes linearly with distance, and *jumps to* β (100%) at any station the student passes through. The dashed red line is the $B_{\min}$ comfort threshold; the optimum keeps every trajectory strictly above it.

```

1  # Trace device battery along a given OD route under a given station set
2  function trace_battery(0, D, stations; beta=BETA_DEFAULT, beta0=BETA0_DEFAULT,
3      Bmin=BMIN_DEFAULT)
4      path = route_nodes_record([0, D])
5      arcs = route_record([0, D])
6      n = length(path)
7
8      cum_dist = zeros(n)
9      B_before = zeros(n) # battery when arriving at node i (before charging)
10     B_after = zeros(n) # battery when leaving node i (after charging, if
11         applicable)
12
13     B_before[1] = beta0
14     B_after[1] = path[1] in stations ? beta : beta0
15
16     for i in 1:n-1
17         d = costs[arcs[i]]
18         cum_dist[i+1] = cum_dist[i] + d
19         B_before[i+1] = B_after[i] - d
20         B_after[i+1] = (path[i+1] in stations) ? beta : B_before[i+1]
21     end
22     return path, cum_dist, B_before, B_after
23 end

```

```

22
23 # Four representative routes
24 demo = [
25     ("Dejope Residence Hall", "College Library"),
26     ("Bakke", "Law School Library"),
27     ("Phillips Residence Hall", "College Library"),
28     ("Smith Residence Hall", "Geology & Geophysics Library"),
29 ]
30
31 stations_C = Set(sel_C_def)
32
33 plt_traj = plot(
34     xlabel = "Distance walked (m)", ylabel = "Device battery B (m range) $\to$%",
35     title = "Battery trajectories under Model C's optimal placement",
36     grid = true, size = (1000, 580), legend = :outright,
37     ylims = (-50, BETA_DEFAULT + 150),
38     yticks = ([0, BMIN_DEFAULT, BETA0_DEFAULT, BETA_DEFAULT],
39              ["0", "B_min (10%)", "$\beta_0$ (60%)", "$\beta$ (100%)"]),
40 )
41
42 # Comfort threshold
43 hline!(plt_traj, [BMIN_DEFAULT], color=:red, linestyle=:dash, linewidth=1.5,
44        label="Comfort threshold (10%)")
45
46 colors = [:steelblue, :darkorange, :green, :purple]
47 for (idx, (O, D)) in enumerate(demo)
48     path, cum, Bb, Ba = trace_battery(0, D, stations_C)
49     label = "$O $\to$ $D"
50
51     # Draw staircase: at each node, vertical segment from B_before to B_after,
52     # then walk to next node
53     xs_line = Float64[]; ys_line = Float64[]
54     for i in 1:length(path)
55         push!(xs_line, cum[i]); push!(ys_line, Bb[i])
56         push!(xs_line, cum[i]); push!(ys_line, Ba[i])
57     end
58     plot!(plt_traj, xs_line, ys_line, color=colors[idx], linewidth=2,
59           label=label)
60
61     # Mark charging events
62     charge_x = [cum[i] for i in 1:length(path) if path[i] in stations_C]
63     charge_y = [Ba[i] for i in 1:length(path) if path[i] in stations_C]
64     scatter!(plt_traj, charge_x, charge_y,
65              marker=:utriangle, markersize=9, color=colors[idx],
66              markerstrokecolor=:black, markerstrokewidth=1, label="")
67
68     # Mark arrival dots at every node
69     scatter!(plt_traj, cum, Bb, marker=:circle, markersize=3.5,
70             color=colors[idx], markerstrokewidth=0, label="")
71 end
72
73 # Annotations explaining markers
74 annotate!(plt_traj, 50, BETA_DEFAULT + 80,
75          text("Delta = charging event (jumps to 100%)", 8, :black, :left))
76 display(plt_traj)
77
78 # Print the textual trace for reference
79 println("\nTrajectory detail (stations: $(join(sort(collect(stations_C)), ",

```

```

    "))) :")
78 println("="^100)
79 for (O, D) in demo
80     path, cum, Bb, Ba = trace_battery(0, D, stations_C)
81     println("\n$O $to$ $D ($round(Int, last(cum))) m total")
82     for i in 1:length(path)
83         star = path[i] in stations_C ? "Battery" : " "
84         below = Bb[i] < BMIN_DEFAULT ? "below threshold!" : ""
85         @printf " %s %-35s B_arrive = %4.0f m (%5.1f%%)$below\n" star
            path[i] Bb[i] (100*Bb[i]/BETA_DEFAULT)
86     end
87 end

```

Code 33: Battery trajectories under Model C placement

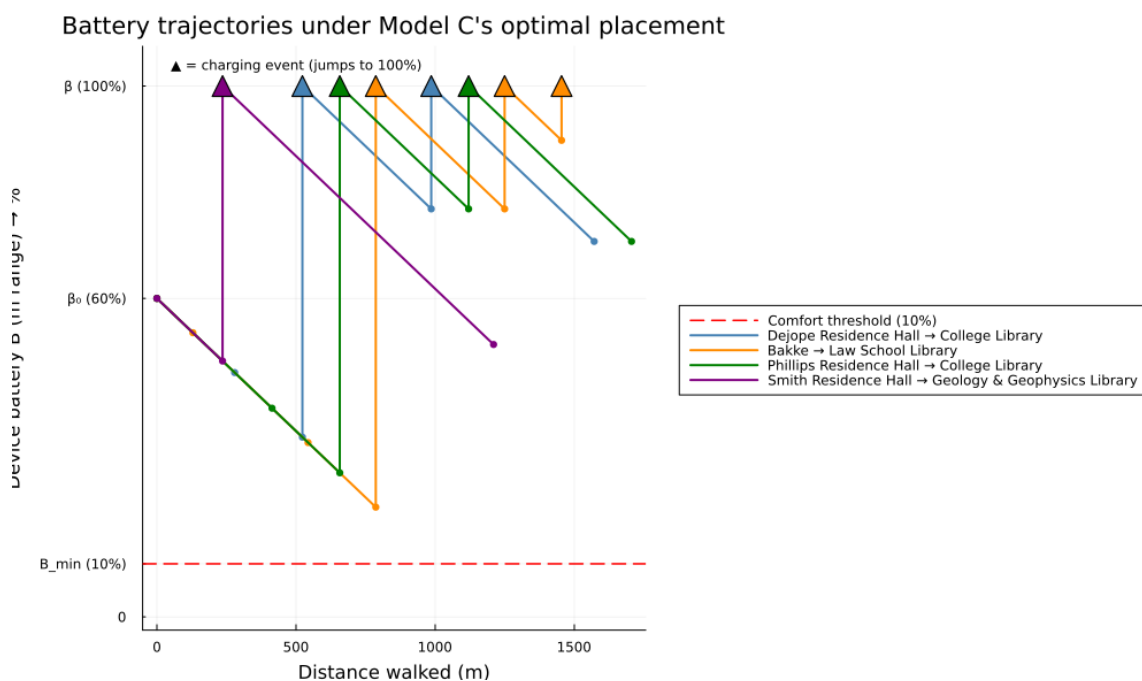


Figure 9. Battery along four representative OD routes under Model C's optimal placement. Triangles mark charging events; the dashed red line is the 10% comfort threshold $B_{\min}$.

The Bakke → Law Library trajectory comes within $\approx 21\%$ of the threshold at Microbial Sciences, a thin margin that motivates adding a redundant station in the final rollout phase (Section 5).

4.8 Limitations and Sensitivity to Assumptions

The model depends on a few key assumptions, and their effects are fairly predictable.

(A1) Shortest-path routing. Real students sometimes detour for coffee or to avoid weather. The Monte Carlo study in Section 4.5 suggests that the placement is not overly sensitive to exact paths: under randomized OD weights, four stations remain dominant, suggesting that the network's hub topology absorbs modest path variation. A more direct test (re-solving Model D with $\pm 10\%$ random perturbations to each route's length) is left as future work but, given the Monte Carlo robustness already observed, is unlikely to change the top-tier recommendations.

(A2) Linear battery depletion. Real device batteries degrade non-linearly with screen-on time. However, our model only uses depletion-versus-distance, not versus-time, and a student walking at constant pace experiences roughly linear depletion in either variable. Using a non-linear discharge curve would shift $B_{\min}$ but not the qualitative conclusions.

(A3) No queueing. This is the most consequential simplification. With $|R| = 36$ daily routes per kiosk in the worst case and a ~ 30 s rental transaction, the steady-state queue is short, but during morning commute peaks (8:30–9:30 am) a single popular kiosk could become a bottleneck. Section 5 sketches an explicit M/M/s extension.

(A4) Full recharge on visit. Power banks dispensed by these kiosks last several hours, so this is realistic for a once-per-trip event but not for repeated visits in quick succession. The model can therefore be interpreted as “one rental per route”, which is consistent with how the underlying app operates.

Cost weights. The installation-cost coefficients c_i span a $\sim 1.5\times$ range (0.9 for academic libraries, 1.4 for residence halls). Since all robust stations identified in the Monte Carlo (Law School, Microbial, Gordon, Van Hise) sits on the lower-cost end of this range, the qualitative recommendations are insensitive to modest perturbations in c_i : doubling residence-hall costs would not affect the top-tier picks, since none of them are residence halls. Network topology, not cost weighting, dominates the optimum—a conclusion that mirrors Chan [3] at the highway scale.

5 Conclusion

5.1 Summary of Findings

This project formulated portable charging-station placement on UW–Madison as five mixed-integer programs of increasing realism and solved them with HiGHS via JuMP. Three findings are robust across modelling choices and demand scenarios.

First, the battery-balance constraint of Equation (2.4) is not cosmetic: it changes the optimum, pushing it toward mid-route relay stations. Under pure set covering (Model B) the optimum buys six stations to cover all 36 routes; the battery-aware model (Model C) covers the same routes with five. The Monte Carlo side-by-side in Figure 6 confirms that the two modes disagree exactly where battery dynamics matter most.

Second, four stations appear consistently in the Monte Carlo results: **Law School Library**, **Microbial Sciences Building**, **Gordon Dining**, and **Van Hise Hall**. The first two are stable under both modelling paradigms (battery-aware and connectivity-only); the latter two are favoured specifically once battery realism is taken seriously.

Third, equity is meaningful but not free: Model E achieves $\eta^* = 0.929$ on the same budget and the same station count as Model D, with AA and RC categories at 100% and one residence-to-academic route uncovered ($13/14 = 0.929$). To push η^* to 1 would require either a fifth station ($\alpha = 5$ is not enough) or a substantially different topology—a useful finding for administrators, who can now decide whether the marginal benefit of full equity is worth the marginal $\sim \$3k$ cost of one more kiosk.

5.2 Recommended Three-Phase Rollout

Combining the diminishing-returns curve of Figure 5, the Monte Carlo robustness in Table 3, and the resilience considerations from Figure 9, we recommend the rollout in Table 4.

Table 4. Recommended three-phase staged rollout.

Phase	Stations added	Cum. budget	Approx. cost	Coverage
Phase 1	Law School Library; Microbial Sciences Building	$\alpha \approx 2.0$	$\sim \$6k$	89%
Phase 2	+ Gordon Dining & Event Center; Van Hise Hall	$\alpha \approx 4.2$	$\sim \$13k$	97%
Phase 3	+ Memorial Library (redundancy / equity)	$\alpha \approx 5.1$	$\sim \$16k$	100%

Phase 1 establishes the network’s two pinch-point stations; Phase 2 adds the two battery-aware mid-route relays that distinguish our approach from naive set covering; Phase 3 adds a redundant academic library to mitigate the single-point-of-failure risk visible at Microbial Sciences in Figure 9. The full deployment attains 100% coverage of every modelled OD pair with five kiosks.

5.3 A Concrete Future Direction

A natural follow-up and one we believe is feasible in practice is to replace the deterministic OD weights w_r with a *two-stage stochastic program* driven by real Wi-Fi-association traces. UW Information Technology operates a managed Wi-Fi network whose anonymized association logs (de-identified MAC→AP→timestamp records, available on request to enrolled students for academic projects) provide a ground-truth empirical distribution over inter-building flows during each hour of the day. Concretely, the proposed extension would:

- (i) construct hourly OD-flow scenarios $\xi^{(s)}$ from the trace data, sampled per weekday and per hour (e.g., 8–9 am Monday, noon–1 pm Wednesday);
- (ii) formulate a first-stage decision $X_i \in \{0, 1\}$ (where to build) and a per-scenario second-stage decision $y_r^{(s)}$ (which routes end up covered under realized demand $\xi^{(s)}$);
- (iii) solve $\min_X \sum_i c_i X_i + \mathbb{E}_\xi [Q(X, \xi)]$ via the sample-average approximation, exploiting the relatively-natural Benders decomposition that our route-by-route constraint structure already supports;
- (iv) validate against an out-of-sample week of trace data to estimate the true coverage probability.

This extension would directly address Limitation (A3) at the demand-modelling level, and the Wi-Fi-trace ingest pipeline is the only piece of new infrastructure required—all other components (Model D, the JuMP code, the HiGHS solver) carry over essentially unchanged. We estimate the work at roughly 40 additional hours beyond what this report contains, well within the scope of a follow-on independent study.

References

- [1] D. Bertsimas and J. N. Tsitsiklis. *Introduction to Linear Optimization*. Athena Scientific, 1997.
- [2] I. Capar, M. Kuby, V. J. Leon, and Y.-J. Tsai. An arc cover–path-cover formulation and strategic analysis of alternative-fuel station locations. *European Journal of Operational Research*, 227(1):142–151, 2013.
- [3] W. Chan. Siting electric vehicle fast charging stations in Wisconsin. CS/ECE/ISyE 524 Project Report, University of Wisconsin–Madison, Summer 2020.
- [4] R. Church and C. ReVelle. The maximal covering location problem. *Papers of the Regional Science Association*, 32(1):101–118, 1974.
- [5] S. L. Hakimi. Optimum locations of switching centers and the absolute centers and medians of a graph. *Operations Research*, 12(3):450–459, 1964.
- [6] M. Kuby and S. Lim. The flow-refueling location problem for alternative-fuel vehicles. *Socio-Economic Planning Sciences*, 39(2):125–145, 2005.